

第三章 用户、组、权限、ACL 与 sudo

身份与权限题要求把账号属性、组成员关系、目录访问和新建对象的继承行为同时配置正确。已有文件可写，不代表新文件会继承相同权限；用户属于组，也不代表当前登录会话已经获得新组。本章用“身份 → 基础权限 → 特殊位与 ACL → 账号策略 → sudo → 实际身份验证”建立闭环。

[概念] 用户由 UID 标识，组由 GID 标识。用户有一个主组和零个或多个附加组；文件只记录一个所有者 UID、一个所属组 GID 和权限模式。名称只是系统文件对数字身份的映射。

[概念] 权限模式按 owner、group、other 三类主体分配 `rwX`。ACL 可为额外用户或组增加条目，并由 mask 限制 named user、named group 与文件 group class 的有效权限；默认 ACL 只影响目录中新创建的后代对象。

[操作语义] `useradd` / `usermod` / `userdel` 管理账号属性，`groupadd` / `groupmod` 管理组；`id` 与 `getent` 查询解析后的身份；`passwd` 与 `chage` 管理口令和期限状态。

[操作语义] `chmod` 修改模式位，`chown` / `chgrp` 修改所有者和组；`getfacl` / `setfacl` 读取和修改 ACL；`visudo` 校验 sudoers 语法，`sudo -l` 显示当前身份可执行的授权。

[知识专题] 账号、主组、附加组与会话身份

① [知识点] 三个账号文件承担不同职责

`/etc/passwd` 保存用户名、UID、主 GID、注释、家目录和登录 Shell；口令哈希及期限字段在 `/etc/shadow`；组名、GID 和显式成员在 `/etc/group`。不要把 `/etc/group` 成员列表为空解释为没人属于该组，因为用户可通过 `passwd` 中的主 GID 归属。

```
getent passwd alice          # 使用系统名称服务查询账号
getent group project         # 查询组及显式成员
id alice                     # 汇总 UID、主组和附加组
```

`getent` 比直接 `grep` 本地文件更符合名称服务语义；考试环境若只用本地账号，结果仍来自相同文件。

② [参数点] 创建与修改的选项必须区分

`useradd -u` 指定 UID，`-g` 指定主组，`-G` 指定附加组，`-d` 指定家目录路径，`-m` 创建家目录，`-s` 指定登录 Shell。`usermod -aG` 追加附加组；漏掉 `-a` 的 `-G` 会把附加组列表替换为给定集合。

```
groupadd -g 3000 project     # 创建指定 GID 的组
useradd -u 2001 -g project -G wheel \
    -m -s /bin/bash alice    # 创建指定身份账号
usermod -aG developers alice # 追加附加组
```

账号创建后用 `id`、`getent passwd` 和 `getent group` 分别核对。不要仅以 `useradd` 退出 0 证明所有 UID、Shell 和组关系都符合题意。

③ [边界] 现有会话不会自动刷新组列表

进程在启动时获得 UID/GID 和附加组。管理员把已登录用户加入新组后，该用户现有 Shell 通常仍使用旧组列表；需要重新登录或启动具有新组身份的会话。排错时分别运行 `id <USER>` 查询账号数据库和在用户会话中运行 `id` 查询进程实际身份。

[Cheatsheet] 账号解析：`getent passwd/group`；最终身份：`id`；创建主组 `-g`、附加组 `-G`；修改时追加必须 `usermod -aG`；组变更后重新建立会话。

[知识专题] 文件与目录上的 rwx、umask 和特殊位

① [知识点] 同一字母在文件与目录上含义不同

普通文件的 `r` 是读取内容，`w` 是修改内容，`x` 是执行；目录的 `r` 是列出目录项，`w` 是创建、删除和改名目录项，`x` 是穿越与访问已知名称。删除文件主要由父目录权限决定，即使文件本身只读，拥有父目录 `wx` 的用户仍可能删除目录项。

权限检查先选择 owner、group 或 other 中唯一一类，不会把多类权限相加。若用户是文件所有者，就只使用 owner 位，即使所属组位更宽。

② [知识点] umask 从创建请求中屏蔽权限

普通文件常以最大请求 `0666` 创建，目录以 `0777` 创建，再由 `umask` 清除位。`umask 0027` 常得到文件 `0640`、目录 `0750`；程序可主动请求更窄权限，因此结果不一定只由 `umask` 决定。

```
umask                                # 查看当前 mask
umask 0027                          # 仅改变当前 Shell 及其后代
```

持久 `umask` 应放在题目指定的 Shell 配置范围，并通过新登录会话创建样本验证；仅运行一次 `umask` 不是持久配置。

③ [知识点] setgid 与 sticky 解决不同目录问题

目录 `setgid (2)` 让新建对象继承目录所属组，适合协作目录；`sticky (1)` 限制可写目录中的删除/改名，通常只允许文件所有者、目录所有者或 `root` 操作。`setuid (4)` 主要作用于可执行文件，使进程获得文件所有者的有效 UID，不应作为普通共享目录方案。

```
chown root:project /srv/project
chmod 2770 /srv/project          # setgid + owner/group rwx
chmod 1777 /srv/ Dropbox        # 公共写入但限制互删
```

④ [验证点] 用实际身份创建对象

`stat -c '%A %a %U %G %n'` 验证目录模式与身份，再用 `sudo -u alice` 和 `sudo -u bob` 创建、读取、修改及删除样本。只看目录位不能证明用户的当前组或父路径权限正确。

[Cheatsheet] 目录访问需逐层 `x`；删除由父目录 `wx` 主导；`umask` 是清除位；协作组继承用 `setgid 2`；公共目录防互删用 `sticky 1`；最终用实际用户验证。

[操作专题] 建立 `setgid` 协作目录并控制新文件权限

① [操作点] 先建立身份与目录所有权

```
groupadd project          # 创建协作组
usermod -aG project alice # 追加成员
usermod -aG project bob
mkdir -p /srv/project
chown root:project /srv/project # 设置目录所属组
chmod 2770 /srv/project      # 组继承与组内完全访问
```

`setgid` 只保证新对象组继承，不保证新文件一定组可写。若用户 `umask` 为 `0022`，新文件通常是 `0644`，组仍不可写。可以为协作用户配置合适 `umask`，或使用默认 ACL 明确新对象的组权限。

② [验证点] 分别验证账号数据库和新对象行为

```
id alice
stat -c '%A %a %U %G %n' /srv/project
sudo -u alice touch /srv/project/alice.txt
stat -c '%A %a %U %G %n' /srv/project/alice.txt
sudo -u bob sh -c 'printf "%s\n" team >> /srv/project/alice.txt'
```

最后一条功能测试能证明 `bob` 当前实际身份可以修改 `alice` 新建的文件；若失败，依次检查 `bob` 的会话组、目录逐层权限、文件 `group` 和有效权限。

[Cheatsheet] 组 → `usermod -aG` → `chown root:<GROUP>` → `chmod 2770`；`setgid` 管组继承，`umask`/默认 ACL 管新对象可写性；用两个真实身份交叉验证。

[操作专题] 使用访问 ACL 与默认 ACL 表达额外授权

① [知识点] `access ACL` 与 `default ACL` 作用时点不同

访问 ACL 描述当前对象谁能访问；目录的默认 ACL 是新建子对象的继承模板，不会自动回溯修改已有文件。`setfacl -m u:alice:rw` 修改访问条目，`-m d:u:alice:rw` 修改默认条目；`-x` 删除指定条目，`-b` 清除扩展访问 ACL，属于可能扩大或收缩权限的动作，不能作为通用排错。

② [操作点] 同时配置当前目录和后续对象

```
setfacl -m u:auditor:rx /srv/project          # 当前目录访问
setfacl -m d:u:auditor:rx /srv/project         # 后代默认条目
setfacl -m d:g:project:rx,d:m:rx /srv/project # 组默认权限与 mask
```

ACL mask 限制 named user、named group 和所属组条目的有效权限。getfacl 中某条权限后出现 #effective:，说明条目写得更宽但被 mask 截断。修改模式位中的 group class 也可能改变 ACL mask。

③ [验证点] 旧对象与新对象分别检查

```
getfacl -p /srv/project
sudo -u alice touch /srv/project/new.txt
getfacl -p /srv/project/new.txt
sudo -u auditor test -r /srv/project/new.txt
```

目录默认 ACL 存在只证明模板已配置；必须创建新对象检查继承，并用目标身份测试功能。已有对象若也需授权，要单独修改其访问 ACL。

[Cheatsheet] 当前访问：setfacl -m u:<USER>:<PERMS>；默认继承加 d:；查询：getfacl；有效权限受 mask 限制；默认 ACL 只在新对象创建时生效。

[操作专题] 配置密码期限、账号锁定与最小 sudo 授权

① [操作点] 区分口令锁定、账号过期和 Shell 禁止登录

passwd -l 锁定口令认证，passwd -u 解锁；它不一定终止 SSH 公钥等其他认证方式。chage -E 设置账号过期日期，-M 最大口令天数，-m 最小天数，-W 到期前警告天数，-d 0 强制下次登录修改口令。

```
chage -M 90 -m 1 -W 7 alice          # 配置口令期限
chage -l alice                        # 查看可读状态
passwd -S alice                       # 查看口令状态摘要
```

/sbin/nologin 阻止交互登录 Shell，但不能被笼统描述为“完全禁用账号”。选择控制入口前先明确题目要求禁止哪种访问。

② [操作点] sudoers 只授予必要命令

主文件是 /etc/sudoers，也可在 /etc/sudoers.d/ 放片段。使用 visudo 或 visudo -cf <FILE> 校验语法和包含关系；文件权限通常为 0440。命令规范应使用绝对路径，并明确是否允许参数；ALL=(ALL) ALL 是广泛授权，不应在只要求管理单个服务时使用。

```
%webops ALL=(root) /usr/bin/systemctl restart httpd, /usr/bin/systemctl status httpd
```

NOPASSWD: 只控制 `sudo` 是否再次询问口令，不改变允许命令范围。通配符和可编辑脚本可能扩大授权，应避免把用户可修改的程序交给 `root` 执行。

③ [验证点] 语法、可见授权与实际执行三层验证

```
visudo -cf /etc/sudoers.d/webops          # 语法
sudo -l -U alice                          # 账号可见规则
sudo -u alice sudo -n /usr/bin/systemctl status httpd # 按授权实际测试
```

若规则不使用 `NOPASSWD`，`-n` 会因需要口令而失败，此时应在合适交互环境验证。`sudo -l` 结果正确仍不能证明命令目标或服务功能正确。

[Cheatsheet] 期限：`chage`；口令状态：`passwd -S/-l/-u`；sudoers 用 `visudo -cf`；授权最小到绝对命令路径；验收：语法 + `sudo -l` + 实际目标身份执行。

[诊断专题] 从 Permission denied 定位身份、路径、模式或 ACL

① [诊断点] 当前进程没有新附加组

账号数据库中的 `id alice` 正确，但 `alice` 已登录 Shell 的 `id` 没有新组，应重新登录或创建新会话。反复 `chmod` 会掩盖真正的会话身份问题。

② [诊断点] ACL 条目存在但 effective 权限不足

运行 `getfacl` 查看 `mask` 和 `#effective:`。修正目标 ACL 或 `mask` 后重新用实际身份测试；不要无条件把 `mask` 设为 `rwX`，因为这会同时放宽整个 `group class`。

③ [诊断点] 文件权限正确但路径仍不可达

用 `namei -l` 检查每级父目录 `x`，再确认 SELinux 等其他安全层。只给最终文件 `chmod 777` 既可能无效又扩大权限。

[Cheatsheet] `id` 查进程身份 → `namei -l` 查父路径 → `stat` 查模式/身份 → `getfacl` 查条目与 `mask` → 目标身份功能测试；不要先全局放宽。

[经典任务] 为项目团队建立共享目录和受控提权

创建 `project` 组以及用户 `alice`、`bob`、`auditor`。`alice` 与 `bob` 属于项目组；`/srv/project` 只允许项目成员创建和修改内容，新对象自动继承 `project` 组且组成员可协作。`auditor` 只能读取和穿越当前目录及后续新对象，不得获得写权限。

`alice` 的 UID 为 2001，登录 Shell 为 `/bin/bash`，口令最大期限 90 天、提前 7 天警告。项目组成员只允许以 `root` 身份查看和重启 `httpd`，不得获得通用 `root` Shell。已有文件不得删除或重建。

验收要求：账号属性、当前会话组、目录模式、已有/新对象 ACL、跨用户写入、`auditor` 只读、`sudoers` 语法与允许/拒绝命令分别有证据。

请先独立完成。参考解答从下一页开始。

[参考解答] 分开配置身份、继承、有效权限和 sudo

① [操作点] 创建账号与期限

```
groupadd project
useradd -u 2001 -g project -m -s /bin/bash alice
useradd -g project -m -s /bin/bash bob
useradd -m -s /bin/bash auditor
chage -M 90 -W 7 alice
```

② [操作点] 配置目录模式和 ACL

```
mkdir -p /srv/project
chown root:project /srv/project
chmod 2770 /srv/project
setfacl -m u:auditor:rx /srv/project
setfacl -m d:u:auditor:r-X,d:g:project:rwX,d:m:rwx /srv/project
```

若已有文件也要求 auditor 读取，应按题意对现有树设置访问 ACL，不能依赖默认 ACL 回溯生效。执行递归修改前先预览对象范围。

③ [操作点] 写入最小 sudoers 片段

```
%project ALL=(root) /usr/bin/systemctl status httpd, /usr/bin/systemctl restart httpd
```

将其保存为 `/etc/sudoers.d/project-httpd`，设置 `0440`，并运行 `visudo -cf`。

④ [验证点] 用不同身份完成验收

```
id alice; id bob; id auditor
chage -l alice
stat -c '%A %a %U %G %n' /srv/project
getfacl -p /srv/project
sudo -u alice touch /srv/project/from-alice
sudo -u bob sh -c 'echo team >> /srv/project/from-alice'
sudo -u auditor test -r /srv/project/from-alice
sudo -u auditor test ! -w /srv/project/from-alice
sudo -l -U alice
```

另以项目成员验证允许命令，并确认未授权命令被拒绝。验证不应改变现有业务服务终态，必要时选择 `status` 作为非破坏性检查。

[Cheatsheet] 身份与期限 → `2770` setgid 目录 → 访问 ACL + 默认 ACL → 新对象交叉测试 → 最小 sudoers → `visudo` + `sudo -l` + 允许/拒绝验证。

[本章收束] 权限结果取决于身份、路径和创建时规则

用户能否访问对象，先取决于进程实际 UID/GID 和附加组，再经过父路径穿越、模式位、ACL mask 以及 SELinux 等其他层。setgid 解决新对象组继承，umask 或默认 ACL 决定创建时权限，它们不能互相替代。

账号策略同样要按作用范围验证：口令锁定不一定阻止公钥登录，nologin 主要阻止交互 Shell，sudoers 授权必须缩小到必要命令。最终使用目标身份创建、读取、修改、删除和提权，才形成完整证据。